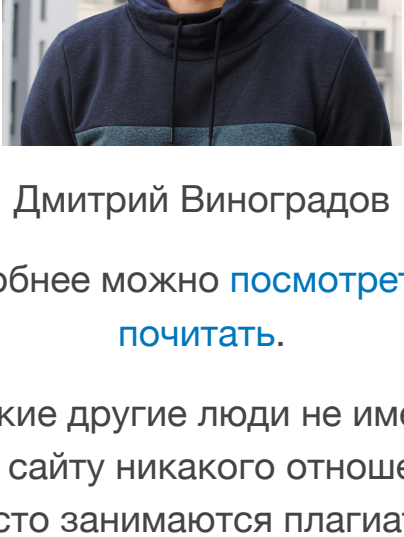


Language



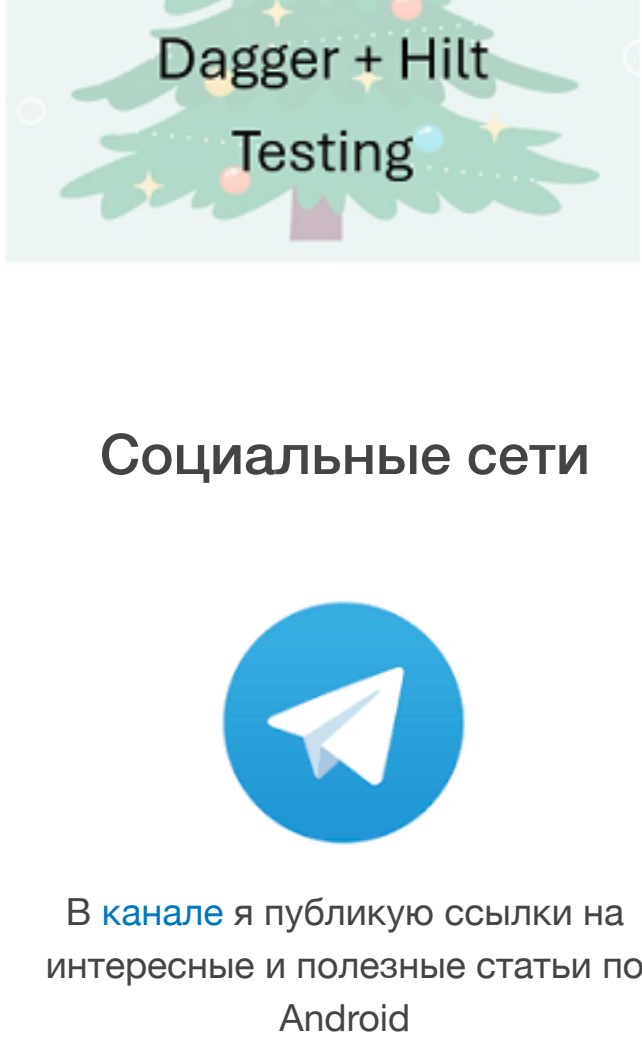
Автор сайта



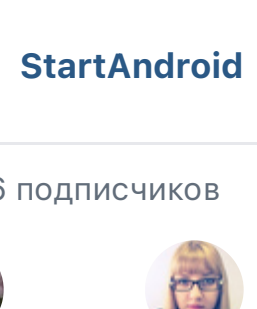
Дмитрий Виноградов

Подробнее можно посмотреть или прочитать

Нашим другим людям не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Социальные сети



В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 1. Введение. Создание проекта. Composable функция

14 декабря 2022

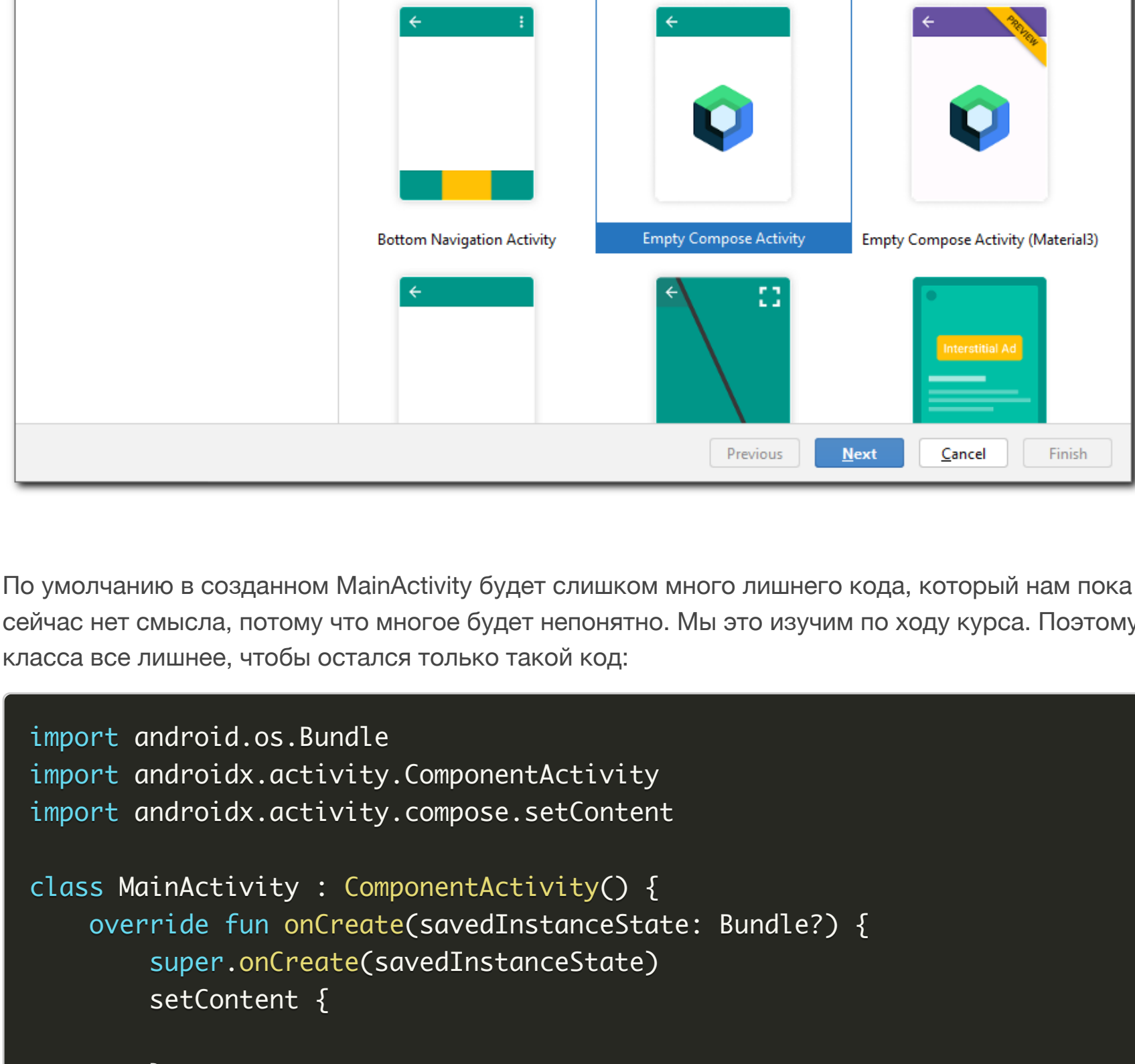
В этом уроке создаем проект для работы с Compose, обсуждаем, что такое Composable функция и создаем свою простую функцию.

Jetpack Compose - новый, декларативный способ формирования UI. И это не просто переход на какой-то очередной Binding. В нем нет layout-архитектуры в XML-файлах. Мы больше не используем findViewById (или ViewBinding), чтобы добраться до View и работать с ним через сеттеры и геттеры.

В целом, такого понятия как View тут больше нет. Чтобы сформировать экран, мы теперь вызываем Composable функции, например: Text(), Button(), CheckBox() и т.п. В эти функции мы передаем данные о том, что надо отобразить и в каком виде. Словами это объяснить непросто, давайте сразу к практике.

Проект

Создаем в студии новый проект. В качестве Activity выбираем Empty Compose Activity:



По умолчанию в созданном MainActivity будет слишком много лишнего кода, который нам пока не нужен. Объяснять его сейчас нет смысла, потому что многое будет непонятно. Мы это изучим по ходу курса. Поэтому пока давайте удалим из этого класса все лишнее, чтобы остался только такой код:

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.compose.setContent

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            // TODO: Add your own Compose UI here
        }
    }
}
```

Метод setContent - это аналог знакомого нам setContentView. Но он от нас ждет не layout файл, а Composable функции. Именно этот метод является переходом из обычного кода в Compose код.

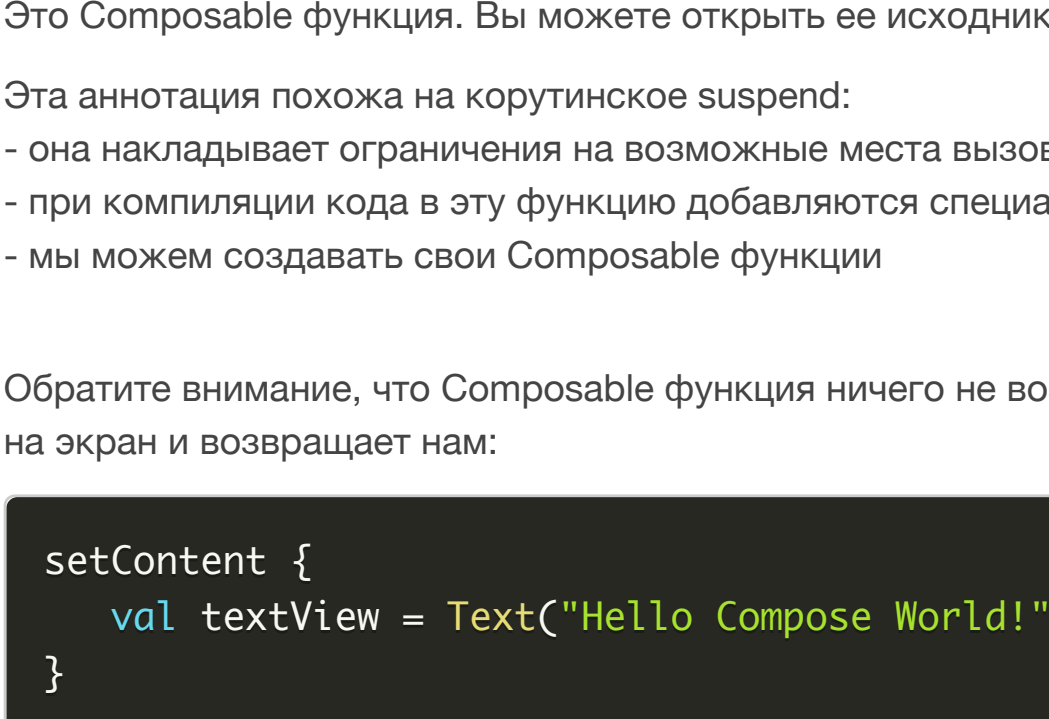
Если мы на экране нашего Activity хотим показать, например, текст, то мы в setContent вызываем Composable функцию Text():

```
import androidx.compose.material.Text

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Text("Hello Compose World!")
        }
    }
}
```

В эту функцию мы передаем нужный нам текст.

Запускаем приложение



Видим текст на экране

Composable

Поговорим немного про функцию Text.

Это Composable функция. Вы можете открыть ее исходники и увидите там аннотацию @Composable.

Эта аннотация похожа на корутиновское suspend:

- она накладывает ограничения на возможные места вызова функции (setContent или другие Composable функции)
- при компиляции кода в эту функцию добавляются специальные системные параметры и вызовы
- мы можем создавать свои Composable функции

Обратите внимание, что Composable функция ничего не возвращает. Т.е. нет такого, что она создает TextView, добавляет его на экран и возвращает нам.

```
setContent {
    val textView = Text("Hello Compose World!")
}
```

Нет. Так это не работает.

Вообще про View можно забыть. Нет больше такого понятия. Остались только Composable функции, которым мы будем сообщать, что именно мы хотим видеть на экране. А они уже под капотом формируют необходимые данные, добавляют их в специальную внутреннюю таблицу (SlotTable), и исходя из этой таблицы система рисует экран. Когда мы захотим обновить данные на экране, нам надо будет снова вызывать Composable функцию с новыми данными. Об этом мы еще подробно поговорим.

Так как мы теперь не можем использовать слово View, чтобы называть тексты, чекбоксы и кнопки, будем использовать понятие Элемент.

Создание Composable функций

Мы можем создавать свои Composable функции. Но это не значит, что тем самым мы создаем свой UI элемент типа текста, чекбокса или кнопки. Это скорее способ организации или группировки нужных нам элементов в одной функции.

Тут можно снова привести аналогию с suspend. У нас есть Retrofit API и Room Dao, которые предоставляют нам suspend функции. Если мы хотим, например, получить данные с сервера и сохранить их в БД, мы в нашем репозитории (или в UseCase) создаем свою suspend функцию и в ней уже комбинируем вызовы API и DB. В итоге наша suspend функция ничего особо и не делает, только вызывает другие suspend функции.

Тут тоже самое. Мы можем, например, создать свою Composable функцию с названием Person, чтобы отображать данные пользователя. В ней мы просто вызываем системные Composable функции Image() и Text() для отображения аватара и имени пользователя.

Т.е. что-то типа такого:

```
@Composable
fun Person(...) {
    Image(...)
    Text(...)
}
```

Это упрощенная и нерабочая версия кода, но смысл она передает

Функция Person в свою очередь также может быть использована в любой другой нашей Composable функции. Например, создаем функцию Team, которая будет отображать данные членов какой-то команды.

```
@Composable
fun Team(...) {
    Text(...)
    Person(...)
    Person(...)
    Person(...)
}
```

Сначала она показывает имя команды с помощью Text, а потом несколько человек из нее, используя Person.

Функция Team тоже может быть вызвана, например, в функции Project, которая отображает данные о каком-то проекте и командах, которые в нем участвуют.

```
@Composable
fun Project(...) {
    Text(...)
    Person(...)
    Team(...)
    Team(...)
}
```

Функция отображает название проекта (Text), руководителя проекта (Person) и команды (Team), которые работают над проектом.

В какой-то момент построения своей иерархии @Composable функций, мы понимаем, что функция Project уже является полноценным экраном, который мы можем использовать для отображения деталей проекта. Такую функцию вполне можно переименовать в ProjectScreen, чтобы по названию сразу было понятно, что речь идет об экране.

Формально она ничем не отличается от любой другой Composable функции. Но в ней будет идти работа с ViewModel, она будет участвовать в навигации и т.п. Обо всем этом еще поговорим в последующих уроках. А пока давайте создадим свою небольшую Composable функцию.

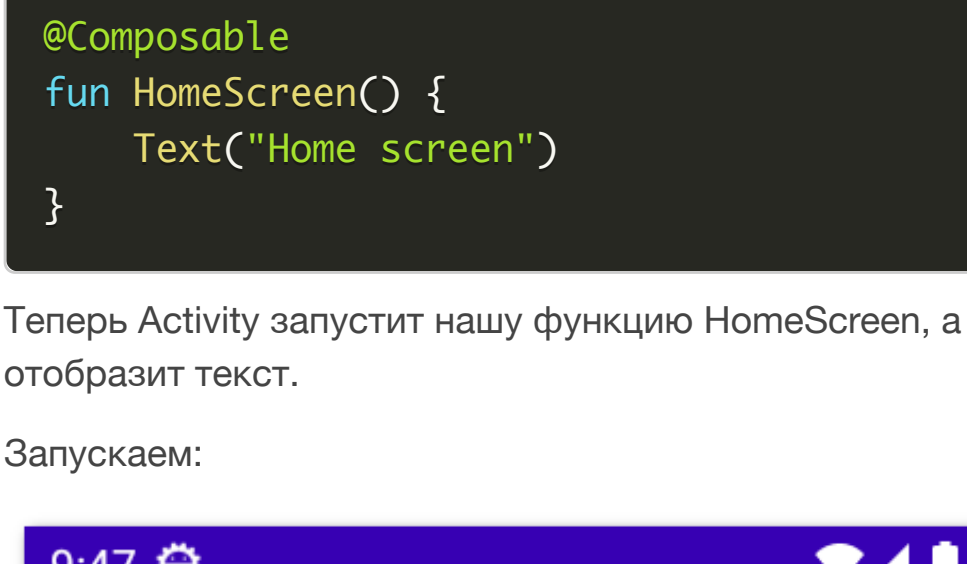
У нас есть MainActivity, которое умеет запускать Composable функции в setContent. Сейчас мы там вызываем функцию Text, чтобы отобразить текст в этом Activity. Но давайте мыслить экранами.

До эпохи Compose мы в Activity использовали фрагменты, например: HomeFragment, OrdersFragment, ContactsFragment. Тем самым мы в одном Activity отображали разные экраны: Главный, Заказы, Контакты.

Теперь же, чтобы отображать экраны, мы будем вместо фрагментов вызывать Composable функции: HomeScreen, OrdersScreen, ContactsScreen. Так же, как и в случае с фрагментами, Navigation будет отвечать за то, какой именно экран сейчас должен отображаться в Activity. Об этом будем говорить в уроке про навигацию.

В данный момент нам хватит только одного экрана, который мы будем вызывать в Activity вручную (без Navigation). Пусть это будет HomeScreen - стартовый экран.

Создадим под него новый файл: HomeScreen.kt



В нем создаем Composable функцию, которая будет нашим стартовым экраном:

```
import androidx.compose.runtime.Composable

@Composable
fun HomeScreen() {
    // TODO: Add your own Compose UI here
}
```

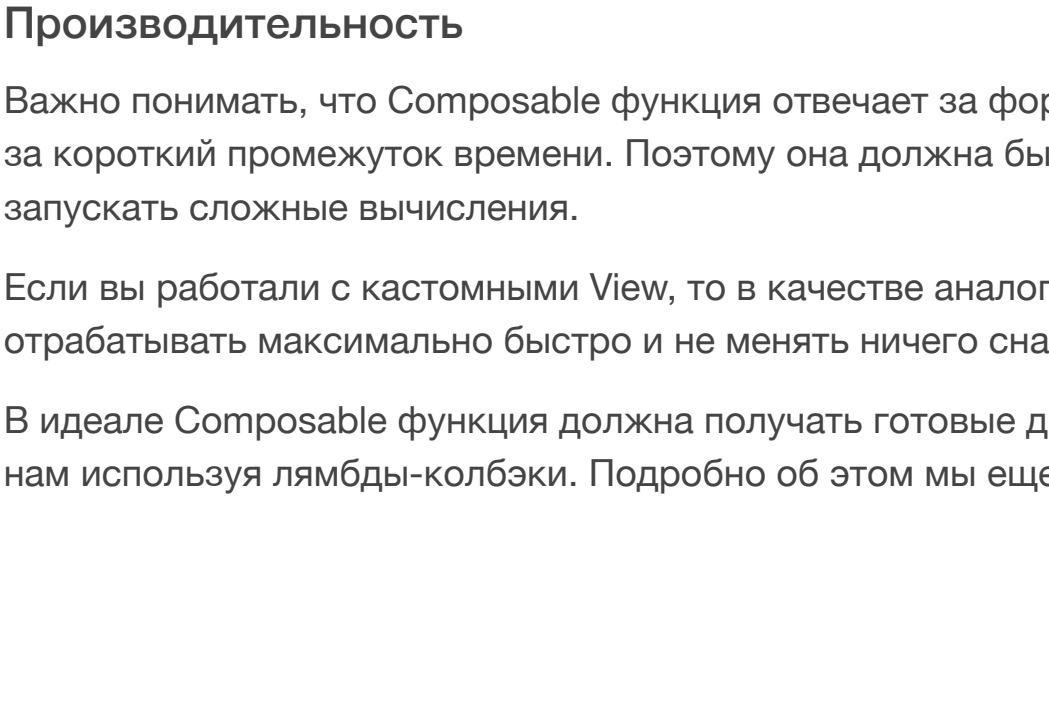
Пока что эта функция пуста. Она не вызывает никакие другие Composable функции, типа Text, CheckBox и пр. Соответственно при вызове этой функции, мы увидим пустой экран.

Давайте проверим. Вызовем эту функцию в setContent в MainActivity:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HomeScreen()
        }
    }
}
```

Таким образом мы попросили Activity отобразить наш стартовый экран.

Запускаем приложение:



Пустой стартовый экран

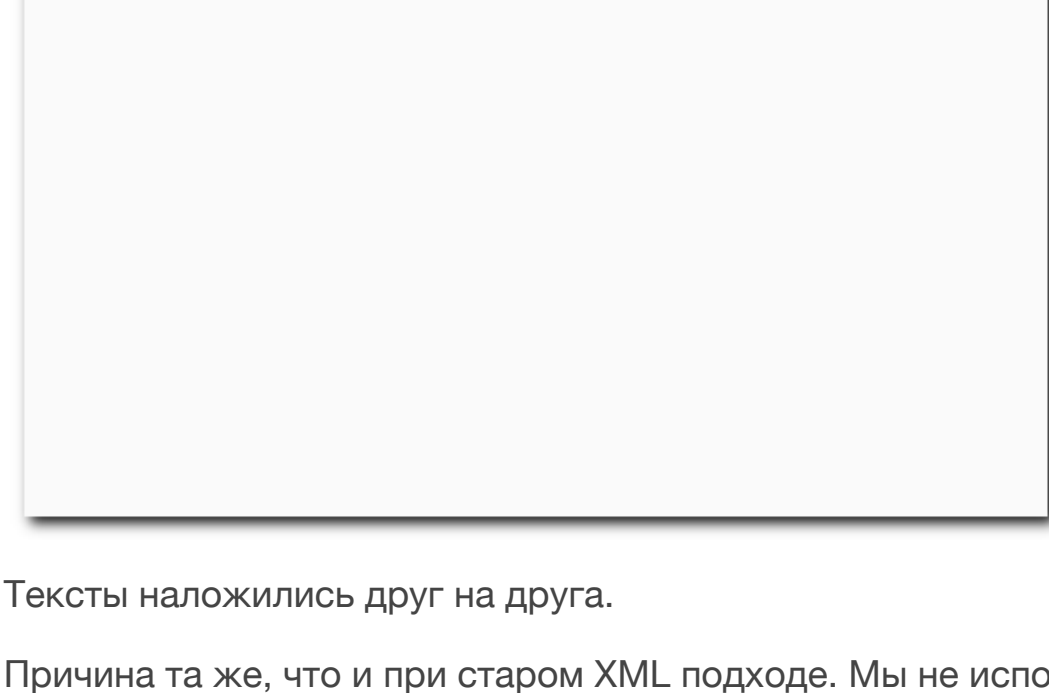
Давайте добавим на него текст:

```
import androidx.compose.runtime.Composable
import androidx.compose.material.Text

@Composable
fun HomeScreen() {
    Text("Home screen")
}
```

Теперь Activity запустит нашу функцию HomeScreen, а в свою очередь запустит системную функцию Text, которая отобразит текст.

Запускаем:



Текст появился на экране.

Мы создали @Composable функцию, которая у нас играет роль очень простого стартового экрана, и попросили MainActivity отобразить этот экран.

Имена функций

Имя Composable функций принято начинать с заглавной буквы.

По имени функции должно быть понятно, что она покажет на экран. И в идеале в ней не должно быть тавтологий, только функциональные. Т.е. не ShowPerson, а просто Person.

Про функции-экраны давайте еще раз проговорим, чтобы не казало, что это какая-то отдельная сущность. Экран HomeScreen в нашем примере не является какой-то особенной Composable функцией. Слово Screen в ее названии тоже не добавляет никакой магии. Просто таким образом удобно называть функцию, в которой мы собрали все то, что хотим видеть на каком-то конкретном экране.

Когда другой разработчик видит название HomeScreen, он сразу понимает что тут зашит контент стартового экрана приложения. В нашем примере этот контент состоит всего из одного UI элемента - Text.

Если вам это имя для экрана кажется неподходящим, используйте свое: HomeEntry, HomeItem, HomeBox, HomeLayout, HomeFragment или просто Home, как вам будет удобно. Я в дальнейших уроках буду придерживаться названия "Screen".

Производительность

Важно понимать, что Composable функция отвечает за формирование UI. Она может быть вызвана системой несколько раз за короткий промежуток времени. Поэтому она должна быть легкой. В ней не надо делать запрос к серверу, писать в БД или запускать сложные вычисления.

Если вы работали с кастомными View, то в качестве аналога можно привести методы onDraw или onMeasure. Они должны отрисовывать максимально быстро и не менять ничего снаружи себя.

В идеале Composable функция должна получать готовые данные на вход и отображать их. А о действиях юзера она сообщает нам используя lambda-коллбеки. Подробно об этом мы еще поговорим в последующих уроках.

Layout

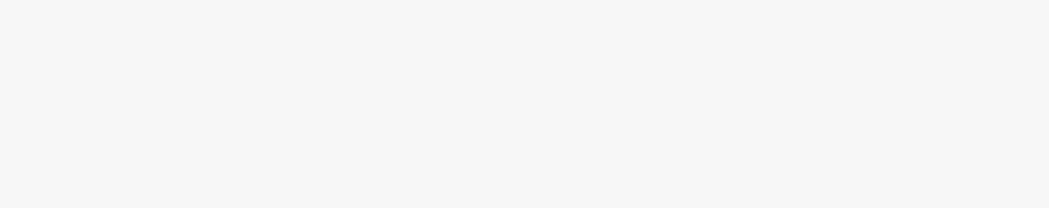
Сейчас экран HomeScreen состоит только из одного текста:

```
@Composable
fun HomeScreen() {
    Text("Home screen")
}
```

Попробуем добавить еще один текст:

```
@Composable
fun HomeScreen() {
    Text("Home screen")
    Text("123456789")
}
```

Запускаем:



Тексты наложился друг на друга.

Причина та же, что и при старом XML подходе. Мы не использовали Layout, чтобы выстроить элементы на экране в нужном нам порядке. Для этого у Compose есть аналоги известных нам LinearLayout, FrameLayout и т.п. В одном из ближайших уроков рассмотрим их.

На этом закончим первый урок. Он несложный, но важный для понимания. Чтобы работать с Compose, необходимо сменить парадигму мышления. Compose UI строится совсем по-другому, чем мы привыкли. Но по своему опыту могу сказать, что к этому достаточно быстро привыкаешь.

Присоединяйтесь к нам в Telegram:

- в канале StartAndroid публикую ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатке решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Фудуиш](#)

Комментарии

RE: Урок 1. Введение. Создание проекта. Composable функция – Клемент 12.01.2023 19:36

Спасибо!

Ответить

Описка – Eugene 18.07.2023 21:49

Скорее всего описка произошла: "на экране, нем надо будет"

Ответить

RE: Описка – Dmitry Vinogradov 21.07.2023 19:02

Пофиксил, спасибо!

Ответить

Может добавить? – Николай 24.07.2023 22:53

@Preview
@Composable
fun DefaultPreview() {
 HomeScreen()
}

Студия требует добавить превью, чтобы отобразить результат

Ответить

RE: Может добавить? – Dmitry Vinogradov 28.07.2023 14:27

Про Preview я рассказываю в следующем урочке. В этом уроке я после примера явно пишу, что надо запустить приложение, чтобы увидеть результат

Ответить

ООП – Илья 07.10.2023 19:51

Почему Composable-функция находится за пределами класса активности? Разве это не противоречит принципу инкапсуляции?

Попробовал перенести их во внутрь класса, работает. Допускаю на таких примерах оно будет работать, но может дальше с чем-то столкнуться.

Ответить

RE: ООП – Dmitry Vinogradov 13.10.2023 13:36

А зачем им быть в Activity? Это отдельные функции-экраны, которые не имеют никакого отношения к Activity. И они могут быть вызваны из разных Activity.

Для фрагментов мы же создаем отдельные экраны. Тут тоже самое. Только не класс, а файл с функцией.

Ответить

👆 Обновить список комментариев

📌 RSS лента комментариев этой записи

Добавить комментарий

Тема

👍👎😄😅😆😇😈😉😊😋😌😍😎😏😐😑😒😓😔😕😖😗😘😙😚😛😜😝😞😟😠😡😢😣😤😥😦😧😨😩😪😫😬😭😮😯😰😱😲😳😴😵😶😷😸😹😺😻😼😽😾😿😺😻😼😽😾😿

Отправить

Спам-бот

